

PKU Captain

面向北京大学学生的桌面 AI 助手——项目作业报告

侯宇泽

石一

邱文晰

2500013225

2500013039

2400010732

@RizzoHou (组长)

@by-lastime

@Q-star17

信息科学技术学院 | 面向对象程序设计实践 | 2026 年 7 月

PKU Captain 是一款面向北大学生的 PyQt6 桌面应用。左边是一块仪表盘，把课表、DDL、课程通知、树洞消息和教务更新聚到一屏；右边是一个能看到工具调用过程的对话式 LLM agent，它靠工具调用和文档库检索，基于真实的校园数据作答，也能把多个步骤串成 workflow。项目代码约一万六千行，62 个模块，450 项自动化测试，四条平行的 OOP 继承体系撑起全部扩展。

1 程序功能介绍

北大学生日常要用的信息是散的。作业在教学网，通知在 Blackboard，办事流程在教务部，闲聊在树洞，每一处都要单独登录，界面各不相同，信息还容易过期。通用大模型又不清楚用户的课表和临近的 DDL。PKU Captain 把这些数据源接进同一个界面，仪表盘一屏总览，agent 用自然语言应答并做一些自动化。产品重心放在仪表盘上，对话 agent 摆在侧栏，负责多步编排和自然语言查询，两者共用同一套后端工具。

1.1 六项核心功能

- 统一仪表盘。今日课表、近期 DDL、未读课程通知、树洞消息、教务更新在一屏里呈现。每张卡片能单独刷新，点某张只刷那张；刷新时用线程池并发拉取，启动耗时取决于最慢的一张卡，不是所有卡相加；卡片的原始数据缓存到磁盘，重开先照上次结果画出来，再静默比对、只重画变了的卡。某张卡失败只影响它自己，不会拖垮其余。
- 对话侧栏。用自然语言查询全部数据，agent 每次工具调用和结果都以内联形式显示出来；回答逐字流式输出；思维链能选是否显示，默认关闭。另有多会话历史、消息复制、上下文用量计量和 LaTeX 公式渲染。
- 工具集。一组能被 agent 调用的工具，覆盖 pku3b（作业、通知、课表、身份）、文档库检索与视觉阅读、持久记忆、日历提醒、P-Lib 资料、教务部资源与更新、树洞检索。每个工具带一份 JSON schema，联网的工具只在在线模式下注册。
- 多步工作流。一个工作流把多次工具调用组合起来，完成较复杂的任务，比如今日简报、周复盘、课程补课。它既能由仪表盘按钮触发，也被包成一个工具，让 agent 在对话里直接发起。
- 文档库，取代了早先的嵌入式 RAG。doc_search 对切分后的 PDF 清单做词法检索，doc_read 用 pdftoppm 把 PDF 渲染成图片交给视觉模型直接读，省掉“PDF 转文本”这一道的信息损耗。

6. 记忆系统。把个人偏好（食堂口味、课程、作息）持久存下来，每次回答时并进系统提示。用户说一句话就能存进去；仪表盘还能调用 LLM 把一整段话拆成几条原子事实分开存储。

1.2 支撑性设计

除核心功能外，还有几处支撑设计。对话大模型抽象成两个能切换的角色，文本角色默认用 DeepSeek，视觉角色默认用 Kimi，端点、模型、密钥都能在设置里改，任何 OpenAI 兼容的接口都能接；切换角色会重置当前对话，并按角色是否支持视觉来开关 `doc_read`。账号统一在一个设置入口里管，分“统一身份·树洞”“P-Lib”“模型配置”“网络代理”四个标签页，一次 IAAA 登录同时供给树洞和 pku3b。网络代理支持跟随系统、直连、自定义三种，保存后对所有 requests（包括 vendored 库和大模型）立即生效。应用默认离线启动，用 `EchoLLMProvider` 和离线工具子集，不配密钥也能跑通整条链路和全部测试，加 `--online` 切到在线。

整个项目的规模大致如下。

指标	数量	指标	数量
src/ 代码行数	≈16,000	OOP 继承体系	4
Python 模块	62	Vendored 校园客户端	4
自动化测试 (pytest)	≈450	可注册工具 (Tool)	13+

2 项目各模块与类设计细节

2.1 总体结构

PKU Captain 分成两层。界面层在 `src/ui/`，后端在 `src/core`、`llm`、`tools`、`workflows`、`rag`。两层之间只有一个工厂做接口，界面从不直接构造具体的 `Provider`、`Tool` 或 `Source`，只调用 `src/core/bootstrap.py` 里的工厂函数 (`build_agent`、`build_source_registry` 等)，拿到抽象类型再交互。这层接口的公共 API、线程模型、事件流和错误约定都写在 `docs/integration_contract_zh.md` 里，是两层协作时都要守的硬约束。`build_agent(offline=True)` 会一键换成离线实现，界面不配密钥也能开发。

2.2 四条平行的 OOP 继承体系

OOP 是这门课的考核重点。项目里所有能扩展的东西都收敛成同一种写法，写一个子类，注册到对应的注册表，不做零散的分发。模块本身没有导入副作用，注册都发生在调用点 `bootstrap.py`，不在 `import` 的时候。加新能力时不改老代码，符合开闭原则。

体系	抽象基类	注册表	典型子类
Tool	Tool / ToolResult	ToolRegistry	Clock（离线示例）、Memory、DocBase、pku3b*、Plib、Dean*、Treehole、CalendarReminder
Workflow	Workflow	WorkflowRegistry	Hello（离线示例）+ WorkflowTool 适配器
LLM	LLMProvider	工厂选择（无注册表）	Echo（离线）、DeepSeek、Kimi
Source	Source	SourceRegistry	Static（离线）、Dean、Calendar

四条体系都按“一个离线参考子类加若干真实子类”来组织，好让内核循环和测试不依赖 API key 或联网就能跑。

Tool 的子类声明 name、description 和 parameters_schema（一份 JSON schema），实现 invoke，返回一个 ToolResult。ToolResult 无论成功失败都会填好结构化结果，还能带一组 images，供视觉模型直接看，doc_read 渲染出来的 PDF 页就走这条路。ToolRegistry 把整个工具集序列化 OpenAI 的工具调用格式，另有一个 unregister，用来按当前模型角色动态开关 doc_read。

LLMProvider 统一了 chat 和 stream_chat 两个接口，流式事件把答案增量和思维链增量分成两路。DeepSeek 实现了真正的 SSE 流式和思维链回放，推理模型要求把上一轮的 reasoning_content 回传，否则会 400；Kimi 负责视觉。两者的 stream_chat 都先按原始字节收流、再统一 UTF-8 解码，免得在分块边界把多字节中文截断。

Workflow 在构造时拿到 ToolRegistry，run 里按名字组合工具调用。它靠注册表的成员检查来判断某个工具在不在，不在或失败就记录下来跳过。agent_callable 这个标志决定它要不要被包成工具暴露给模型。

Source 按各自的 refresh_interval 轮询，fetch 返回一串 Chunk，后面的采集流程统一处理。

2.3 Agent 内核与回合循环

Agent.turn() 是对话的主循环，最多迭代 8 轮。

```
user msg → Conversation.add_user → loop (≤8)
  取快照，把记忆折叠进领头的 system 消息（逐轮刷新，不写入 Conversation）
  → llm.stream_chat → 事件 reasoning_delta • assistant_delta • llm_response
  → 若有 tool_calls, tool_call → tool.invoke → tool_result (逐个)
    ↳ 若 ToolResult.images, 在所有结果后补一条多模态 user 消息（视觉模型读图）
  → context_usage → (无 tool_calls) → final
```

这里有一处关键设计，记忆是折叠进快照的，不写进对话本身。bootstrap 让 MemoryTool 和 Agent 共用同一个 MemoryStore 实例，每一轮把当前记忆经 render_memory_context() 合并进快照副本领头的 system 消息，所以对话中途写入的记忆能立刻反映出来，留存的对话历史又保持干净。工具产生的错误串在工具边界（redact.py）先做凭据脱敏，再进入对话，注入或持有的密钥进不到 LLM 请求，也落不到会话文件里。

2.4 支撑模块与运行时数据流

后端还有一批各管一摊的服务类。Conversation 管历史，MemoryStore 和 MemoryLearnService 管记忆和 LLM 事实抽取，CredentialStore 是 secrets/ 唯一的写入者和状态来源，network 里的 ProxyConfig 和 apply_proxy 管进程级代理，SessionStore 和 SessionTitler 管多会话的持久化和自动命名，DashboardCache 把卡片原始数据落盘，VisionRouter 把培养方案一类的问题自动切到视觉角色。界面这边按集成契约用三个 QThread，AgentWorker 跑对话回合，DashboardWorker 刷新仪表盘，WorkflowWorker 跑工作流，另有一个 QThreadPool（tool_call_worker.run_async）接住对话框里的阻塞调用，保证窗口一直响应；取消回合用的是 threading.Event，不用 PyQtSlot，因为 worker 这时正阻塞在回合里。

对话之外还有三条数据流，写法上都遵循后台线程、错误隔离、缓存优先这几条。仪表盘刷新时，

DashboardWorker 用 ThreadPoolExecutor 把选中的工具并发跑起来，用 as_completed 逐张卡发结果，启动耗时就是最慢那张卡的时间；每张卡的异常都单独吞掉、单独上报，一张卡崩不会连累别的。DashboardCache 把每张卡的原始数据存到磁盘，启动先照着画，再静默刷新，只重画数据变了的卡；刷新范围也分级，单张卡的按钮只刷自己，表头的刷新才重载全部。

会话这块，SessionStore 在每回合结束或关闭时把历史写到 data/sessions/ 下，且只在第一次真正对话之后才写；SessionTitler 用一个非思考模式的小模型异步起名；重开时 restore_conversation 会把不完整的 tool-call 尾巴丢掉，免得一个中断的回合让会话重开时一直 400。代理这块，账号中心保存网络设置后 apply_proxy 直接写进程环境变量，之后所有 requests 调用马上生效，不用重启。

pku3b、P-Lib、树洞、教务部这四个校园客户端都用 git subtree 放进 vendor/，映射成顶层包 (pypku3b、plib_cli、dean、treehole)，在进程里直接调用，没有子进程、没有兄弟 .venv、没有外部二进制，应用完全自包含。其中 pypku3b 用 requests 加 bs4 把原来 Rust 版 pku3b 用到的那部分（教学网 Blackboard 加信息门户，走 IAAA 单点登录）纯 Python 重写了一遍，还把 TLS 固定到系统信任库，因为 certifi 验不过 course.pku.edu.cn 的 GlobalSign 证书链。

2.5 测试与可验证性

大约 450 项 pytest 覆盖了记忆、会话、工具、仪表盘并发刷新、视觉注入、上下文计量这些核心路径。GUI 流程用一个不依赖第三方库的 headless 框架来跑真实的 worker 线程，并对渲染出来的状态做断言。凡是改动 core、llm 或工具调用的线格式，还要跑一次 scripts/smoke_deepseek.py，做一次真实的 DeepSeek 往返冒烟。python -m src.cli 是走同一个 build_agent 循环的命令行 REPL，用来核对界面和后端之间的接口是否对得上。

3 小组成员分工情况

项目采取组长吸收整合、成员自由认领的方式。周任务列成待办清单，不预先分配负责人，谁认领未勾选的条目、做完就在后面加上自己的 GitHub 用户名。三个人的主要贡献如下。

成员	学号	GitHub	主要负责
侯宇泽	2500013225	@RizzoHou	组长；整体架构与集成契约、Agent 内核、LLM Provider、工具框架、vendored 客户端、跨任务整合与评审
石一	2500013039	@by-lastime	界面主体，含主窗口、对话面板、工具调用可视化、仪表盘、流式与思维链渲染、多会话交互
邱文晰	2400010732	@Q-star17	知识与工具层，含文档库检索与视觉阅读、记忆系统、Source 采集、部分校园工具、pytest 测试体系

侯宇泽（组长，@RizzoHou）负责整个项目的骨架和接口。他搭起四条 OOP 抽象基类和注册表、Agent 的回合循环、build_agent 工厂和集成契约；写了 DeepSeek 和 Kimi 两个 Provider 的 SSE 流式和思维链回放；把 pku3b 等四个校园客户端用 git subtree 收进仓库，改成在进程里直接驱动。此外，跨任务的整合、PR 评审的协调、发布前的凭据审计也都由他承担。

石一（@by-lastime）独立完成了界面这一层的大部分工作，分量不轻。他从空窗口起步，搭出 MainWindow 的外壳和“仪表盘 | 对话”两栏布局；实现了对话面板的收发、Markdown 和 LaTeX

渲染、消息复制，以及“每段助手回复各占一个气泡”的分段逻辑；把工具调用做成内联可见(InlineToolCall)，让 agent 每次调工具和结果都摆在用户眼前；仪表盘的今日课表、近期 DDL、通知等卡片和配套的 PKU3bCourseTableTool 也是他做的。流式输出、思维链窗口的自适应尺寸、多会话历史开关这些又琐碎又容易出错的 Qt 细节，他都一一收拾干净。界面是用户接触产品的第一层，他对渲染顺序、线程安全和视觉一致的把控贯穿了整个 UI。

邱文晰 (@Q-star17) 负责知识、工具和测试这几块，覆盖面很广。她做了取代嵌入式 RAG 的文档库，把 doc_search 词法检索、doc_read 视觉阅读的整条链路以及配套的 DocBaseReader 服务都实现了出来；搭了持久化记忆系统，包括 MemoryStore 的内容哈希入库和 MemoryLearnService 的 LLM 事实抽取，让个人偏好能并进每次回答；接入并调试了教务部资源等好几个校园信息源和工具。她还独立搭起了项目的 pytest 测试体系，如今这 450 项测试覆盖记忆、会话、工具、仪表盘并发刷新、视觉注入等核心路径，是“main 始终能演示”的底子。测试和知识这两块往往最容易被忽视、也最费功夫，她在这里的投入把项目的稳定性稳稳撑住了。

4 项目总结与反思

先说一句实话。这个项目挺大的，大约一万六千行代码、62 个模块、四条 OOP 继承体系、四个 vendored 校园客户端、450 项测试，还牵涉流式 LLM、视觉多模态、并发刷新、进程级代理、凭据脱敏这些细节。三个本科生在一门课的时间里，靠手一行行敲，是敲不出这个规模的。所以我们不绕弯子，这里大部分代码是 vibe 出来的，由大模型在我们的引导下生成。

我们觉得，这个项目里真正花了我们心思、也真正属于我们的地方是另一件事。在写代码越来越便宜的今天，怎么把 vibe 出来的代码做成有质量的代码。当敲代码的成本趋近于零，工程的难点就从“怎么把它写出来”挪到了“怎么保证一大堆自己没逐字读过的代码是对的、能维护、查得动”。DEVCHANGELOG、VERIFICATION、TASTES 这几样东西，还有我们给它们各写的一个 skill，就是冲着这件事去的。

具体做法是把注意力从代码本身挪开，分摊到几件事上，每件事对应一份和技术栈无关的文字，再由一个项目级 skill 按变更类型自动维护。CLAUDE.md 记那些绝对不能破坏的规则，比如记忆的 store 必须共享、某处只能累积不能替换，它每次开工都会被载入。ARCHITECTURE.html 是系统的结构图，让不写代码的人也能看清哪块和哪块通信、边界在哪，只在结构变了的时候才动。

DEVCHANGELOG 记的是当时为什么这么选，和记“交付了什么”的 CHANGELOG 分开，同一次改动可以在两份文件里各留一条，各说一面。VERIFICATION 把验证从“读代码”搬到“看行为”，写成一份不写代码的人也能照着点、照着复制的手工验证步骤，专门补 GUI、真实网络往返、凭据处理这些自动测试盖不到的地方。TASTES 记的是编码口味，都是从这个仓库自己的教训里攒出来的，比如子类加注册、导入别带副作用、按原始字节解码中文，不是照搬网上那些通用说教。

流程上还有一条。任何不小的、会动结构的改动，都要先用文字把计划讲清楚，说明要动哪些文件、打算怎么做、可能弄坏什么、有哪些假设，让“架构师”先审计划。审的是计划，不是写完之后的 diff。要不要新增一个服务、要不要加一个全局，这类边界决定被顶到计划这一层来定；用哪个 Qt 信号、调哪个 API 这种细节留在代码里。而贯穿全项目的“子类加注册”，本身就把“加新能力”约束成了一种可预料、可枚举、可测的动作。

做下来慢慢体会到一点。一个大模型生成的大系统之所以容易失控，症结常常在于没有人替它维护一套人看得懂、核得动的记录。结构、决策、规则、口味、验证散在各处，还互相串味。代码由机器写

这件事本身倒不是问题。我们的应对，是把这几样东西一件件理清楚、各自分开，再给每一样配一个自动维护它的 skill，这样即便不写代码，也能审出整个系统当下的状况。

这套做法也有它盖不住的地方，我们照样写下来。比如凭据泄露，行为测试看不出来，应用一边漏一边照常跑，只能靠发布前人工过一遍。又比如大模型写的代码，局部看没问题，却可能在系统层面埋下隐蔽的状态耦合，我们靠“共享单例”“只累积不替换”这类约定反复堵这种洞。把这些边界写清楚，也算这套方法的一部分。

回头看，这个项目对我们更像一次试验，在写代码几乎不要钱的时候，怎么让这些几乎白来的代码仍然可信。我们交出的是一个能跑的应用，但更想请老师看的，是上面这套做法，还有那几个自动维护它的 skill。它们比代码本身更能说明我们在这门课里到底做了什么。